2026 第五届“未来杯”创新大赛（校赛）

织序 AgentLoom

可验证的多 Agent 团队编排与调度平台

项目白皮书 · 赛题一（Agent 编排创新应用）

多 Agent 系统正在从演示走向生产。织序把“团队”拆成一张可编排的图，把“思考”做成可挂载的节点能力插件，把“结果”交给核查器与 Self-Loop。

> **Agent 是主机，插件是卡，画布是工作台。**

参赛赛道	赛题一：Agent 编排创新应用
项目名称	织序 AgentLoom：可验证的多 Agent 团队编排与调度平台
团队名称	【待填：队伍名称】
提交日期	2026 年 11 月

目录

摘要

一、项目概述

二、解决方案

三、应用成效

四、商业模式

五、团队介绍

六、发展规划

附录 A：演示入口

附录 B：关键指标口径

附录 C：风险与应对

摘要

多 Agent 系统正在从演示走向生产，但真正落地时普遍遇到三道坎：团队结构写死、思考过程黑盒、结果无法验证。织序 AgentLoom 把“团队”拆成一张可编排的图，把“思考”做成可挂载的节点能力插件，把“结果”交给核查器与 Self-Loop。

平台的差异化不在于“支持多少个 Agent”，而在于三个可演示的动作：

1. 把一张“发散思考”插件挂到任意 Agent 节点上，该节点展开稳健、进取、反方三条思维分支；
2. 裁判按证据覆盖、假设可检验性、行动性收敛出一个主判断；
3. 核查员逐条核对结论与证据，不通过就带着问题清单回到写作节点重写。

我们以“AI 眼镜产业链投研”为首发场景，跑通了知识库、8 角色团队、三路并行研究、节点级发散、裁判收敛、引用核验与 Self-Loop 的完整闭环。内测运行中，报告第一轮核查 85%，发现 2 条问题；回环后第二轮达到 100%。

一、项目概述

1.1 背景：多 Agent 的三道坎

第一道坎：团队写死。 多数 Agent 产品把角色、工具、流程硬编码在代码或提示词里。用户想加一个“风险审查员”，往往要改代码；想换一个行业，往往要重做一套系统。

第二道坎：思考黑盒。 “让模型多想几步”通常只是一句提示词。用户不知道它想了哪几个角度、每个角度质量如何、为什么最后选了这个结论。

第三道坎：结果不可验。 Agent 说“报告完成了”，但报告里的数字是否有依据、引用是否真的支持结论、哪些地方是假设，缺少可复现的核查。失败之后，系统通常只是重试，而不是带着问题修正。

这三道坎决定了：多 Agent 的门槛不在“能不能跑”，而在“能不能被控制、被解释、被验收”。

1.2 项目定位

织序 AgentLoom 是一个**验证优先的多 Agent 编排调度平台**：

> Agent 是主机，插件是卡，画布是工作台。

- **团队层**：8 个角色组成 DAG，支持并行、依赖、回环。
- **认知层**：节点可以挂载能力插件，改变“这个节点怎么想”。
- **验证层**：核查器把结论与证据对齐，失败驱动回写。
- **观察层**：调度、分支、回环、Token 全部事件化，可实时看、可回放。

1.3 应用范围

平台本身不绑定行业。首发场景选择“产业投研”，因为该场景同时具备三个特征：

- 资料密集，需要知识库与引用；
- 结论开放，需要多视角与裁判；

- 错误代价高，需要事实核查。

可迁移的场景包括：企业战略研究、政策研究、竞品分析、软件交付、客服工单、招投标文件编制等。迁移时替换的是 Agent 角色、知识源与提示词，插件与运行时保持不变。

1.4 核心优势

1. **节点级能力插件**：不是“换一个 Agent”，而是“给同一个 Agent 换一种思考方式”。
2. **验证驱动回环**：核查结果不是展示分数，而是下一轮写作的输入。
3. **可观察的调度**：优先级、并发、重试、回环都以事件呈现，能解释“为什么先跑这个”。
4. **知识库 grounding**：结论带稳定引用，引用可回到原文。
5. **可演示的闭环**：拖卡、展开、裁判、核查、回环，3 分钟能讲清楚。

二、解决方案

2.1 总体架构

层	组成	职责
控制台	React 19、Vite、React Flow	总览、知识库、团队、画布、运行、评测
API	Fastify、REST、SSE	运行、知识库、Agent、工作流接口与实时事件
编排运行时	调度器、DAG 引擎、Self-Loop、核查器	依赖执行、并发调度、回环与验收
数据与模型	SQLite、中文 BM25、Mock / DeepSeek	运行轨迹、证据检索、模型调用

2.2 领域模型

实体	说明
Project	一个研究项目，绑定知识库、团队、工作流
KnowledgeDoc / Chunk	资料与可检索片段，片段是引用的最小单位
Agent	角色、模型、温度、系统提示词、工具、插件
Plugin	能力插件，声明分类、配置与运行时语义
Workflow	节点与边组成的 DAG，边区分 flow 与 loop
Run / NodeRun	一次运行与每个节点的每次尝试/回环
RunEvent	append-only 调度、分支、核查事件

2.3 节点能力插件

插件不挂在团队上，而是挂在 Agent 节点上。它只负责三件事：

1. **展开**：把一个输入展开成 N 个分支；
2. **约束**：给每个分支不同的视角、证据或温度；

3. **收敛**：内部裁判收敛，或把多个候选暴露给下游/人工。

MVP 已落地的两个运行时插件：

发散思考（`cognition.diverge`） 在首席分析师节点展开稳健、进取、反方三条分支。每条分支有独立立场与评分，裁判选择主判断，所有候选保留在节点输出中，用户可展开查看。

引用核验（`verify.citations`） 抽取报告中的事实性结论，检查引用是否存在、数字是否与证据一致、语义是否重叠。核查不通过时输出问题清单，触发 Self-Loop。

插件库中还包含知识检索、自我批判、多角色辩论、数值计算。检索已通过研究员角色落地；批判/辩论/计算是下一阶段扩展，MVP 不把它们伪装成已完成能力。

2.4 编排与调度

DAG 编排。 节点是非 loop 边的依赖单位；loop 边只用于表达回环，不参与拓扑排序。

优先级。 调度器计算每个节点到汇点的最长路径深度，深度越大越优先。三路研究同深度，按画布坐标稳定排序。

并发。 默认最大并发 3，三路研究可以并行；分析、写作、核查串行。

失败处理。 节点失败最多重试一次；预算超限在模型调用前终止；取消在调度循环边界生效。

动态性。 运行时的节点状态、分支、重试、回环都会改变就绪队列；调度器每轮重新计算，而不是一次性静态排程。

2.5 Self-Loop 与结果验证

核查器不是“让模型给自己打分”，而是可复现的规则核查：

1. 抽取事实性章节中的结论，过滤标题、引用块与列表编号；
2. 从 `[M2]` 这样的标记定位证据；
3. 忽略年份，检查结论中的数字是否都能在证据原文中找到；
4. 对结论与证据做中文分词，要求至少 2 个实词重叠；

5. 计算支持率，低于阈值则输出问题清单。

回环流程：

1. verifier 不通过，当前 NodeRun 标记为 waiting，保存核查结果；
2. verifier 自身与 writer 重置为 pending；
3. feedback 设为问题清单；
4. loopCount + 1，写入 loop.triggered 事件；
5. writer 读取 feedback，生成 v2；
6. verifier 再核查，直到通过或达到最大回环次数。

达到最大回环次数后，即使未通过也继续终稿，但保留低分并标记“需要人工复核”。这避免了“无限重试”与“假装通过”两个极端。

2.6 知识库与引用

- 按 `##` 章节切分，单节超过 340 字按句切到约 300 字；
- SQLite 存储片段，检索时在内存计算 BM25；
- 中文使用 `Intl.Segmenter` 词级分词 + 双字回退，兼容英文/数字；
- 研究员按市场/产业链/竞争三个角度检索目标文档，生成 M/L/C 前缀的稳定引用；
- 报告中的引用可点击查看原文片段。

2.7 交互与可观察性



图 1 运行控制台：实时图谱、节点轨迹与调度事件同屏

- **总览**：项目、团队、最近运行、关键指标。
- **知识库**：资料入库、检索测试、片段查看。
- **Agent 团队**：8 个角色、插件库、节点级插件开关。
- **编排画布**：拖动节点、查看配置、挂载/卸载插件、运行前检查。
- **运行页**：实时图谱、节点轨迹、调度事件、报告、核查与回环。
- **评测**：运行历史、回环前后对比、质量/成本指标。

2.8 技术栈与部署

层	技术
前端	React 19、Vite、React Flow、React Markdown
后端	Node.js 24、Fastify、SSE
存储	SQLite（`node:sqlite`）
检索	自研中文 BM25
模型	Mock / DeepSeek OpenAI-compatible
运行	本地单机，默认只监听 127.0.0.1

部署只需要 Node.js 22.19+ 与 pnpm：`pnpm build && pnpm start`。

2.9 与同类产品的差异

维度	LobeHub	DeepSeek Harness	织序 AgentLoom
核心抽象	助手/插件市场	插件化 Harness	节点能力插件 + 可验证团队
编排	以对话/助手为中心	代码 Agent 运行时	可视化 DAG + 回环
调度	通用任务调度	运行时工具调用	关键路径优先级 + 并发 + 预算
验证	以人工反馈为主	以工具结果为主	引用/数字/语义核查 + Self-Loop
差异点	生态与助手	工程化运行时	认知插件 + 验收闭环

我们不试图替代它们，而是补上“团队级编排 + 节点级认知 + 结果级验收”这一层。

三、应用成效

3.1 验证场景

任务：生成一份面向投资决策的 AI 眼镜产业链研究报告，重点回答市场空间、价值链分配、竞争格局与主要风险。

知识库：6 篇模拟整理资料，35 个片段，覆盖市场、产业链、竞争、技术、风险、投资框架。

团队：研究规划师、市场研究员、产业链研究员、竞争情报员、首席分析师、报告撰写师、事实核查员、主编。

3.2 运行过程

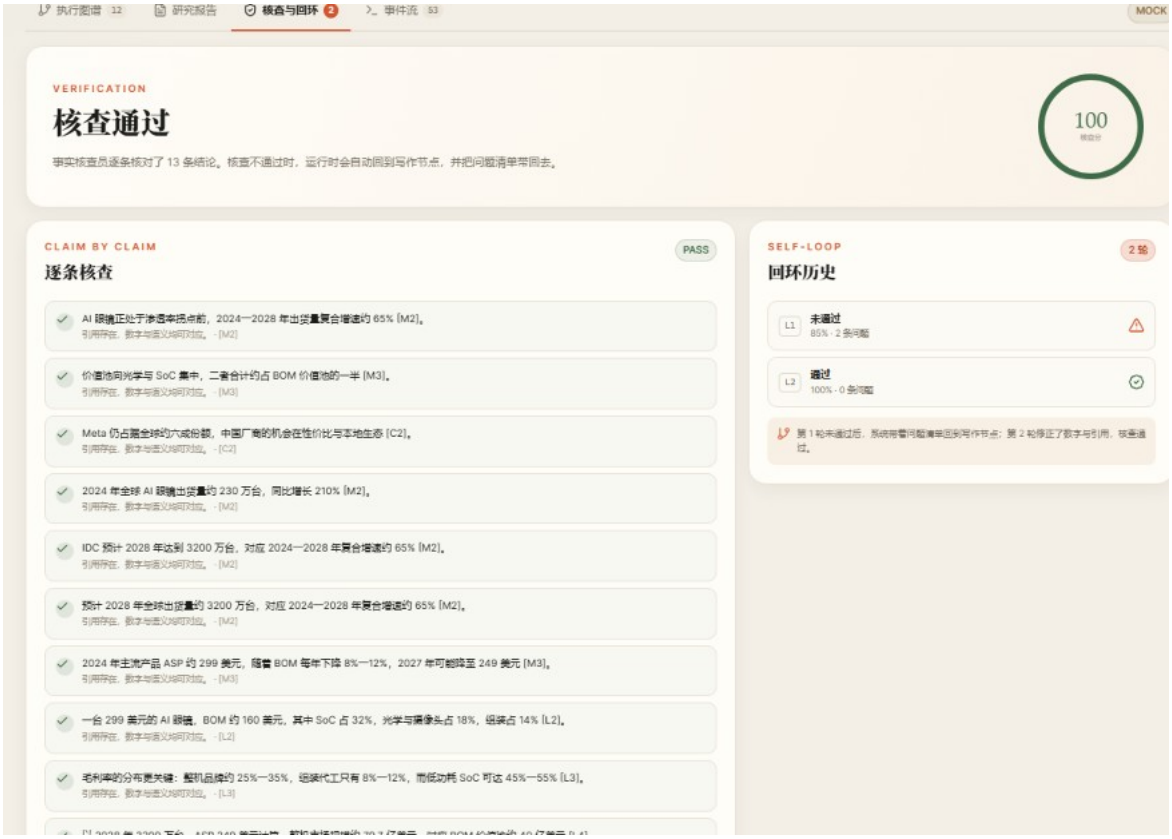


图 2 核查与 Self-Loop：第一轮 85%，第二轮 100%

1. 输入研究任务，规划师拆出 3 个研究问题；
2. 市场、产业链、竞争三路并行检索（最大并发 3）；
3. 首席分析师展开稳健、进取、反方三条分支；

4. 裁判选择稳健情景作为主判断；
5. 撰写师生成 v1 报告；核查员给出 85%，发现 2 条问题：2028 年出货量写成 5000 万台（证据为 3200 万台），“2027 年第一轮洗牌”缺少引用；
6. Self-Loop：带着问题清单回到写作，撰写师生成 v2，修正数字、补充引用；
7. 核查员第二轮 100% 通过，主编终稿。

3.3 结果数据

指标	第一轮	第二轮
核查分	85%	100%
问题数	2	0
核查结论数	13	13
回环次数	—	1
运行耗时（Mock）	—	约 3.3 秒
Tokens（Mock 计量）	—	24,318

> 说明：以上为 Mock 演示模式的真实运行记录，Token 与耗时为模拟口径；Live 模式的成本与延迟需在接入 DeepSeek 后重新测量。

3.4 应用价值

对研究者：把“找资料、做分析、写报告、查引用”串成一条可复盘的生产线，减少重复劳动。

对团队负责人：能看到每个节点的输出、证据与核查问题，知道报告哪里不可信，而不是只看一个最终文档。

对平台开发者：新增一个能力，不需要改整个流程；把插件挂到节点上即可。

3.5 可移植性

平台的插件与行业解耦：

- 发散插件不关心“AI 眼镜”还是“储能”，它只负责展开视角；

- 核查插件不关心报告主题，它只负责结论与证据对齐；
- 换行业时替换 Agent 角色、知识库与提示词，运行时不变。

下一步将用“软件交付”或“客服工单”做第二个团队，验证同一套插件的迁移成本。

四、商业模式

4.1 目标客户

1. **高校与科研机构**：研究团队、智库、实验室，需要可复现的研究流程。
2. **券商、咨询、投资机构**：产业研究、尽调、投委会材料，对引用与核查敏感。
3. **企业战略/市场部门**：竞品分析、政策研究、年度规划。
4. **AI 平台团队**：希望把 Agent 能力封装成内部可复用的团队模板。

4.2 价值主张

- 把“一个人写报告”变成“一条可验收的生产线”；
- 把“Agent 黑盒”变成“可观察、可干预的节点”；
- 把“提示词复用”变成“团队模板 + 能力插件复用”。

4.3 收入模式

模式	内容	假设价格
私有化部署	本地部署 + 知识库 + 团队模板	8–20 万元/年
团队模板订阅	行业模板、插件、评测集	1–3 万元/年
实施与陪跑	知识库治理、流程定制、培训	5–10 万元/项目
插件分成	第三方插件/模板上架	平台抽成 20%–30%

4.4 市场测算（假设）

> 以下为早期测算假设，需在试点中校验，不作为对外承诺。

- 目标早期客户：1000 家（高校研究团队 + 中小型券商/咨询/企业战略部）。
- 客单价假设：10 万元/年。
- SAM 假设：1000 × 10 万 = 1 亿元/年。
- 3 年 SOM 假设：30 家付费 × 10 万 = 300 万元/年。

- 成本结构：研发、实施、算力、渠道；MVP 阶段以项目制交付为主。

4.5 推广路径

1. **校内试点**：以本校研究团队/创新创业学院为第一个场景，积累运行数据与案例。
2. **行业 PoC**：选择 1–2 个投研/咨询客户，做 4–6 周付费验证。
3. **模板化**：把投研、政策研究、竞品分析做成标准团队模板。
4. **渠道**：与高校实验室、咨询公司、AI 服务商合作分成。
5. **生态**：开放插件契约，让第三方开发者贡献“认知插件”和行业模板。

4.6 竞争壁垒

- **协议壁垒**：节点能力插件的契约与运行时语义，不是简单 prompt 模板。
- **数据壁垒**：每个团队模板附带评测集与核查规则，越用越准。
- **迁移成本**：团队模板、知识库、插件组合形成工作区资产。
- **先发场景**：在“可验证投研”这个细分场景建立案例与口碑。

五、团队介绍

> 以下成员信息按报名表填写；当前为占位，提交前替换为真实姓名、专业、履历与分工。

角色	姓名	专业/年级	核心履历	项目分工
队长	【待填】	【待填】	【待填】	产品定位、架构、答辩
算法	【待填】	【待填】	【待填】	调度器、核查器、评测
前端	【待填】	【待填】	【待填】	编排画布、运行控制台
后端	【待填】	【待填】	【待填】	API、知识库、模型网关
运营	【待填】	【待填】	【待填】	场景、商业模式、材料

5.1 团队优势

- **工程落地能力**：从 0 到 1 完成可运行平台，而不是停留在原型图。
- **交叉背景**：AI、软件工程、产业研究的组合，能同时理解模型与场景。
- **产品意识**：把“节点插件”抽象成可演示、可解释的交互，而不是堆功能。

5.2 分工与协作

- 双周迭代：每周一个可演示增量，周末做一次完整回归。
- 代码与文档同仓：架构、演示脚本、评测口径统一维护。
- 答辩前完成 3 次全流程彩排，准备 Mock 与 Live 两套演示。

六、发展规划

阶段	时间	目标
MVP	2026.11 上旬	跑通知识库、8 角色、发散、核查、回环
校赛	2026.11 中旬	完成白皮书、PPT、视频与现场演示
试点	2026.12–2027.02	2 个校内/企业场景，积累真实数据
产品化	2027.03–2027.06	插件契约、团队模板、PDF 入库、权限沙盒
商业化	2027 下半年	私有化部署 + 模板订阅 + 渠道合作

附录 A：演示入口

- 本地启动：`pnpm install && pnpm dev`
- Web：http://127.0.0.1:5173
- API：http://127.0.0.1:8787
- 重置演示数据：`pnpm reset`

附录 B：关键指标口径

- **核查分**：被证据支持的结论数 / 核查结论总数。
- **一次通过率**：首次核查即达到阈值的运行占比。
- **回环次数**：verifier 节点 `iteration` 的最大值。
- **Token**：Mock 模式为模拟计量；Live 模式取模型返回的 usage。
- **耗时**：从 run.started 到 run.completed。

附录 C：风险与应对

风险	应对
多分支成本高	只在关键节点挂载；设置预算与最大分支数
裁判不可靠	规则核查 + 人工复核；保留候选与理由
场景迁移难	插件与行业解耦；先做 2 个场景验证
演示翻车	Mock 兜底；提前录屏；准备离线数据
原创性质疑	运行时、调度器、核查器自研；DSH 仅作为思想参考